OBJECT ORIENTED ANALYSIS AND DESIGN (CT 651)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 4: Implementation

Syllabus Hours: 15 hrs | Marks Weight: 27 marks

CHAPTER 4 SYLLABUS TOPICS

- 4.1 Programming and Development Process
- 4.2 Mapping Design to Code
- 4.3 Creating Class Definitions from Design Class Diagrams
- 4.4 Creating Methods from Collaboration Diagram
- 4.5 Updating Class Definitions
- 4.6 Classes in Code
- 4.7 Exception and Error Handling

SECTION 1: Programming and Development Process

Exam Ref: 2082 Kartik Q10 [2+3+5], 2081 Chaitra Q11, 2081 Ashwin Q10, 2080 Chaitra Q10, 2080 Ashwin Q10, 2079 Ashwin Q10, 2079 Chaitra Q10

Q1. What is meant by 'mapping design to code' in the OO development process? Explain the programming and development process in OOAD. [5] (2082 Kartik, 2081 Ashwin)

Answer:

Programming and Development Process in OOAD

In OO development, the transition from design to code is guided by the artifacts produced during OOD — primarily the Design Class Diagram (DCD) and Collaboration/Interaction Diagrams. 'Mapping design to code' means systematically translating these design artifacts into actual class definitions, method bodies, and object interactions in a programming language such as Java or C++.

The Development Process — Steps:

Step	Activity	Input	Output
1. Review Design	Study all design diagrams before coding. Understand class responsibilities and object interactions.	DCD, Collaboration Diagrams, Use Cases	Clear understanding of what to code
2. Create Class Skeletons	For each class in DCD: create the class file with all attributes and method signatures (no bodies yet).	Design Class Diagram	Skeleton .java / .cpp files
3. Map Methods from Collab Diagrams	For each collaboration diagram: implement the method bodies by following the message flows.	Collaboration Diagrams	Implemented method bodies
4. Implement Constructors	Write constructors to initialize object state. Use Creator pattern guidance.	DCD attribute section	Working constructors
5. Add Associations	Implement associations as reference fields (attribute visibility) or parameters (parameter visibility).	DCD relationships	Fields and associations in code
6. Update and Refine	Add access modifiers, handle edge cases, apply patterns (Singleton, etc.), update definitions.	Review of method bodies	Refined, complete class files
7. Exception Handling	Add try-catch blocks, define custom exception classes, ensure robust error handling.	Non-functional requirements	Robust, error-safe code
8. Test	Unit test each class, integration test object collaborations.	Use case scenarios	Verified, tested code

Key Principle — Minimal Gap:

The goal of OO implementation is to keep the 'representational gap' between design and code as small as possible. Because OO design uses the same concepts as OO programming (classes, objects, inheritance, polymorphism), the translation from DCD to code is nearly mechanical and direct.

Q2. What is the role of the Design Class Diagram and Collaboration Diagram in the implementation phase? How does a programmer use these diagrams during coding? [5] (2080 Chaitra, 2081 Chaitra)

Answer:

Diagram	Role in Implementation	How Programmer Uses It
Design Class Diagram (DCD)	Provides the STATIC blueprint: class names, attributes (with types and visibility), method signatures, and relationships.	1. Create one class file per class in DCD. 2. Declare all attributes with their exact types. 3. Write method signatures exactly as shown. 4. Implement inheritance and interfaces from generalization/realization.
Collaboration Diagram	Provides the DYNAMIC blueprint: shows which object calls which method, in what order, for each use case operation.	1. For each numbered message in diagram: write that line of code in the calling method. 2. Follow sequence numbers to determine order of statements. 3. Use creation messages to write object instantiation.
SSD (System Sequence Diagram)	Shows the system-level API — the public operations the system must support.	Guides writing of the Controller class public interface.
Use Case Text	Describes preconditions, success guarantee, and extensions.	Guides where to add validation, guard conditions, and exception handling.

SECTION 2: Creating Class Definitions from Design Class Diagrams

Exam Ref: 2082 Kartik Q11 [10], 2081 Chaitra Q12 [10], 2081 Ashwin Q11 [10], 2080 Chaitra Q11 [10], 2080 Ashwin Q11 [10], 2079 Ashwin Q11 [10], 2079 Chaitra Q11 [10], 2078 Chaitra Q11 [10], 2078 Baishakh Q11 [10], 2077 Chaitra Q11 [10] — HIGHEST MARKS QUESTION

Q3. Explain the process of creating class definitions from a Design Class Diagram. What are the mapping rules? [5] (2082 Kartik partial, 2080 Chaitra)

Answer:

Mapping Rules — DCD to Java Class Definitions

DCD Element	Java/Code Implementation	Example
Class name	public class ClassName { ... }	public class Sale { ... }
Abstract class	public abstract class ClassName { ... }	public abstract class Payment { ... }
Interface (<<interface>>)	public interface InterfaceName { ... }	public interface IPayable { ... }
Private attribute (- name: Type)	private Type name;	private Money total;
Public attribute (+ name: Type)	public Type name;	public String title;
Protected attribute (# name: Type)	protected Type name;	protected Date date;
Method signature (+ method(p:T): R)	public R method(T p) { ... }	public Money getTotal() { ... }
Abstract method	public abstract ReturnType method();	public abstract boolean pay();
Generalization (A inherits B)	public class A extends B { ... }	public class CashPayment extends Payment { ... }
Realization (A implements I)	public class A implements I { ... }	public class CashPayment implements IPayable { ... }
Attribute association (A --> B)	private B bRef; // or List<B> refs	private Sale currentSale; // in Register
Composition (A *-- B)	private List<B> parts = new ArrayList<>(); // created in A's constructor	private List<SaleLineItem> lineItems = new ArrayList<>();
Aggregation (A <>-- B)	private B part; // B created externally, passed in	private ProductCatalog catalog; // passed to Register
Multiplicity 1..*	private List<B> items; or private B[] items;	private List<SaleLineItem> lineItems;
Multiplicity 0..1	private B optional; // may be null	private Payment payment; // null until paid

Step-by-Step Mapping Process:

- Step 1: For each class in DCD, create a public class (or abstract class or interface).
- Step 2: Declare all attributes with their exact access modifiers and types.
- Step 3: Write all method signatures exactly as shown in DCD.
- Step 4: For each 'extends' (generalization), add 'extends ParentClass'.
- Step 5: For each 'implements' (realization), add 'implements InterfaceName'.
- Step 6: For each association, add a reference field (attribute visibility) or leave for method parameters.

Q4. Given the following Design Class Diagram for a POS System, write complete Java class definitions for: Register, Sale, SaleLineItem, ProductDescription, Payment, CashPayment. [10] (Most repeated — 2082, 2081, 2080, 2079, 2078, 2077)

Answer:

Java Class — Register (Facade Controller)

```
public class Register {
    // Attribute Visibility — Register permanently holds Sale and Catalog references
    private Sale currentSale;
    private ProductCatalog catalog;
    private Store store;

    // Constructor
    public Register(Store store) {
        this.store = store;
        this.catalog = store.getProductCatalog();
    }

    // Controller methods — handle all system operations from SSD
    public void makeNewSale() {
        currentSale = new Sale();  // Creator: Register creates Sale
    }

    public void enterItem(ItemID id, int qty) {
        ProductDescription spec = catalog.getSpecification(id); // Expert: catalog
        currentSale.makeLineItem(spec, qty);  // delegate to Sale
    }

    public Money endSale() {
        return currentSale.getTotal();  // Expert: Sale knows total
    }

    public void makePayment(Money cashTendered) {
        currentSale.makePayment(cashTendered);  // delegate to Sale
    }
}
```

Java Class — Sale

```
import java.util.ArrayList;
import java.util.List;
import java.util.Date;

public class Sale {
    private Date date;
    private boolean isComplete;
    private List<SaleLineItem> lineItems = new ArrayList<>(); // Composition
    private Payment payment;  // 0..1 — null until makePayment called

    public Sale() {
        this.date = new Date();
        this.isComplete = false;
    }

    // Creator: Sale creates SaleLineItem (Sale CONTAINS SLI)
    public void makeLineItem(ProductDescription spec, int qty) {
        SaleLineItem sli = new SaleLineItem(spec, qty);
        lineItems.add(sli);
    }

    // Information Expert: Sale has all lineItems, so it calculates total
    public Money getTotal() {
        Money total = new Money(0);
        for (SaleLineItem sli : lineItems) {
            total = total.add(sli.getSubtotal()); // delegate to each SLI
        }
        return total;
    }
```

```
    // Creator: Sale creates Payment (Sale RECORDS Payment)
    public void makePayment(Money cashTendered) {
        this.payment = new CashPayment(cashTendered);
        becomeComplete();
    }

    public void becomeComplete() {
        this.isComplete = true;
    }

    public boolean isComplete() { return isComplete; }
    public Payment getPayment() { return payment; }
    public List<SaleLineItem> getLineItems() { return lineItems; }
}
```

Java Class — SaleLineItem

```
public class SaleLineItem {
    private int quantity;
    private ProductDescription description; // Association --> ProductDescription

    public SaleLineItem(ProductDescription desc, int qty) {
        this.description = desc;
        this.quantity = qty;
    }

    // Information Expert: SLI has qty and ProductDescription (with price)
    public Money getSubtotal() {
        return description.getPrice().times(quantity);
    }

    public int getQuantity() { return quantity; }
    public ProductDescription getDescription() { return description; }
}
```

Java Class — ProductDescription

```
public class ProductDescription {
    private String description;
    private Money price;
    private ItemID itemID;

    public ProductDescription(ItemID id, Money price, String desc) {
        this.itemID = id;
        this.price = price;
        this.description = desc;
    }

    public Money getPrice() { return price; }
    public String getDescription() { return description; }
    public ItemID getItemID() { return itemID; }
}
```

Java Abstract Class — Payment (Polymorphism + Protected Variations)

```
public abstract class Payment {
    protected Money amount;

    public Payment(Money amount) {
        this.amount = amount;
    }

    public Money getAmount() { return amount; }
```

```
    // Abstract method — each subclass implements differently (Polymorphism)
    public abstract boolean pay();
}
```

Java Class — CashPayment (extends Payment)

```
public class CashPayment extends Payment {
    private Money cashTendered;
    private Money changeGiven;

    public CashPayment(Money cashTendered) {
        super(cashTendered);  // call parent constructor
        this.cashTendered = cashTendered;
    }

    @Override
    public boolean pay() {
        // Cash payment is always immediately successful
        return true;
    }

    public Money calculateChange(Money saleTotal) {
        this.changeGiven = cashTendered.minus(saleTotal);
        return changeGiven;
    }

    public Money getChangeGiven() { return changeGiven; }
}
```

SECTION 3: Creating Methods from Collaboration Diagrams

Exam Ref: 2082 Q12 [10], 2081 Chaitra Q13 [10], 2081 Ashwin Q12 [10], 2080 Chaitra Q12 [10], 2080 Ashwin Q12 [10], 2079 Ashwin Q12 [10], 2079 Chaitra Q12, 2078 Chaitra Q12, 2078 Baishakh Q12, 2077 Chaitra Q12

Q5. Explain how methods are created from Collaboration Diagrams. What is the mapping process? [5] (2082 Kartik, 2079 Ashwin)

Answer:

Mapping Process — Collaboration Diagram to Method Bodies

Every message in a Collaboration Diagram corresponds to a method call in the implementation. The sender's method body contains the code for that message, and the receiver object must have that method defined. The sequence numbers (1, 1.1, 1.2) directly translate to the order of statements in code.

Mapping Rules:

Collaboration Diagram Element	Java Code Equivalent
Numbered message: 1: makeNewSale()	First statement in the calling method body.
Sub-message: 1.1: <<create>>()	First sub-statement — object instantiation: new Sale()
Nested call: 1.2.1: getSubtotal()	Called within the loop/sub-call that triggered it.
<<create>> message to ObjectB	new ObjectB(...) — constructor call
Return value: spec = getSpecification(id)	ProductDescription spec = catalog.getSpecification(id);
Loop (*): 1.3.1*: getSubtotal()	for (SaleLineItem sli : lineItems) { sli.getSubtotal(); }
Guard [condition]: [overdue] sendFine()	if (isOverdue()) { sendFine(); }
Self-call: Sale calls itself	this.methodName() or just methodName()
Link to object with parameter visibility	Method parameter — object passed in
Link to object with attribute visibility	Direct field reference — this.fieldName

Step-by-Step Translation:

- Step 1: Find the top-level message (numbered 1) — this is the method being implemented.
- Step 2: For each sub-message (1.1, 1.2...) of message 1, write a statement in the body of message 1's method.
- Step 3: For each nested sub-message (1.1.1), write code inside the method of message 1.1's receiver.
- Step 4: <<create>> messages become constructor calls: new ClassName(args).
- Step 5: Messages with return values become assignments: result = obj.method(args).
- Step 6: Loop messages (*) become for-each loops over collections.

Q6. Given the Collaboration Diagram for enterItem(itemID, qty) of the POS system, write the complete method implementations derived from the diagram. [10] (2082 Kartik, 2081 Chaitra, 2080 Chaitra, 2078 Baishakh)

Answer:

Collaboration Diagram Messages — enterItem():

Msg #	Sender	Message	Receiver
1	:Register	enterItem(itemID, qty)	:Sale
1.1	:Sale	spec = getSpecification(itemID)	:ProductCatalog
1.2	:Sale	makeLineItem(spec, qty)	:Sale (self)
1.2.1	:Sale	<<create>>(spec, qty)	:SaleLineItem
1.2.2	:Sale	add(sli)	:Sale.lineItems

Implementation — Register.enterItem():

```
// Register class — method derived from message '1' in collaboration diagram
public void enterItem(ItemID itemID, int qty) {
    // Message 1: Register delegates to currentSale
    currentSale.makeLineItem(                 // calls Sale.makeLineItem
        catalog.getSpecification(itemID),     // msg 1.1: gets ProductDescription
        qty
    );
}
```

Implementation — ProductCatalog.getSpecification():

```
// Message 1.1: Sale asks ProductCatalog for product specification
public ProductDescription getSpecification(ItemID itemID) {
    // Information Expert: ProductCatalog has all ProductDescriptions
    return specifications.get(itemID);
}
```

Implementation — Sale.makeLineItem():

```
// Message 1.2: Sale creates a new SaleLineItem and adds to list
public void makeLineItem(ProductDescription spec, int qty) {
    // Message 1.2.1: Creator — Sale creates SaleLineItem
    SaleLineItem sli = new SaleLineItem(spec, qty);
    // Message 1.2.2: add sli to lineItems collection
    lineItems.add(sli);
}
```

Implementation — SaleLineItem constructor (from <<create>> message):

```
// Message 1.2.1: <<create>>(spec, qty) maps to constructor
public SaleLineItem(ProductDescription spec, int qty) {
    this.description = spec;
    this.quantity = qty;
}
```

Q7. Derive method implementations from the Collaboration Diagram for makePayment(cashTendered) of the POS system. [10] (2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra)

Answer:

Collaboration Diagram Messages — makePayment():

Msg #	Sender	Message	Receiver
1	:Register	makePayment(cashTendered)	:Sale
1.1	:Sale	<<create>>(cashTendered)	:CashPayment
1.2	:Sale	becomeComplete()	:Sale (self)
1.3	:Sale	total = getTotal()	:Sale (self)
1.3.1*	:Sale	subtotal = getSubtotal()	:SaleLineItem (each)
1.3.1.1	:SaleLineItem	price = getPrice()	:ProductDescription
2	:Register	printReceipt(sale)	:Printer

Implementation — Register.makePayment():

```
public void makePayment(Money cashTendered) {
    // Message 1: Register delegates payment to currentSale
    currentSale.makePayment(cashTendered);
    // Message 2: Register asks Printer to print receipt
```

```
    printer.printReceipt(currentSale);
}
```

Implementation — Sale.makePayment():

```
public void makePayment(Money cashTendered) {
    // Message 1.1: Creator — Sale creates CashPayment
    this.payment = new CashPayment(cashTendered);
    // Message 1.2: Self-call — mark sale as complete
    becomeComplete();
}
```

Implementation — Sale.getTotal():

```
public Money getTotal() {
    // Message 1.3: Sale calculates total — Information Expert
    Money total = new Money(0);
    // Message 1.3.1*: loop over all SaleLineItems
    for (SaleLineItem sli : lineItems) {
        total = total.add(sli.getSubtotal()); // msg 1.3.1
    }
    return total;
}
```

Implementation — SaleLineItem.getSubtotal():

```
public Money getSubtotal() {
    // Message 1.3.1.1: SaleLineItem asks ProductDescription for price
    return description.getPrice().times(quantity);
}
```

Implementation — ProductDescription.getPrice():

```
public Money getPrice() {
    return this.price;  // Simple getter — Information Expert
}
```

SECTION 4: Updating Class Definitions

Exam Ref: 2082 Q13a [5], 2081 Chaitra Q14, 2080 Chaitra Q13, 2080 Ashwin Q13, 2079 Ashwin Q13, 2078 Chaitra Q13, 2077 Chaitra Q13

Q8. What is meant by 'updating class definitions' during implementation? Why is it necessary? Explain the process with examples. [5] (2082 Kartik, 2081 Chaitra)

Answer:

Updating Class Definitions — Overview

During implementation, as method bodies are written and tested, it is common to discover that the original Design Class Diagram (DCD) needs to be updated to reflect new knowledge. Updating class definitions is the process of refining class code based on insights gained during coding — adding missing attributes, adjusting method signatures, adding helper methods, refining types, etc.

Reasons for Updating Class Definitions:

Reason	Description	Example
Missing attributes discovered	When writing method bodies, you realize a class needs an attribute not in the original DCD.	Sale needs a 'taxRate' field to calculate taxes. Not in original DCD — add it.
Method signature refinement	Return types or parameter types need adjusting based on implementation reality.	getTotal() return type changed from double to Money class.
Helper/private methods needed	Complex methods are split into smaller private helper methods for readability.	Sale.getTotal() uses private helper calculateLineItemTotal().
Collection type specification	DCD shows 1..* but doesn't specify collection type; implementation must choose.	List<SaleLineItem> vs ArrayList<SaleLineItem> vs Set<SaleLineItem>.
Pattern implementation	Patterns like Singleton require adding private constructor, static instance field.	ProductCatalog adds: private static ProductCatalog instance; private constructor.
Interface implementation discovered	During coding, you realize a class should implement an interface for flexibility.	Payment implements IRefundable; add refund() method.
Constructor parameters refined	Constructors need different parameters than originally designed.	SaleLineItem constructor needs (ProductDescription, int) not just (int).

Example — Updating Sale Class with Tax Calculation:

Original DCD — Sale class (before update):

```
public class Sale {
    private Date date;
    private boolean isComplete;
    private List<SaleLineItem> lineItems;
    public Money getTotal() { ... }
}
```

Updated Sale class (after discovering tax requirement during implementation):

```
public class Sale {
    private Date date;
    private boolean isComplete;
    private List<SaleLineItem> lineItems = new ArrayList<>();
    private double taxRate;           // NEW — discovered during implementation
    private Store store;               // NEW — needed for tax lookup

    // UPDATED — now calculates with tax
    public Money getTotal() {
        return getPreTaxTotal().times(1 + taxRate);
    }

    // NEW private helper method (discovered during implementation)
    private Money getPreTaxTotal() {
        Money total = new Money(0);
```

```
        for (SaleLineItem sli : lineItems) {
            total = total.add(sli.getSubtotal());
        }
        return total;
    }
}
```

Q9. What is a Singleton class? Write the complete Java implementation of the ProductCatalog class as a Singleton. [5] (2081 Ashwin, 2079 Ashwin, 2080 Ashwin)

Answer:

Singleton Class — ProductCatalog Implementation

ProductCatalog is implemented as a Singleton because the entire POS system should share one instance of the catalog. Multiple instances would waste memory and could cause data inconsistency.

```
import java.util.HashMap;
import java.util.Map;

public class ProductCatalog {
    // Step 1: Private static instance — the single shared object
    private static ProductCatalog instance = null;

    // Internal storage for product specifications
    private Map<ItemID, ProductDescription> specifications;

    // Step 2: Private constructor — prevents external instantiation
    private ProductCatalog() {
        specifications = new HashMap<>();
        loadFromDatabase(); // load catalog from DB on creation
    }

    // Step 3: Public static getInstance() — provides global access
    public static ProductCatalog getInstance() {
        if (instance == null) {              // lazy initialization
            instance = new ProductCatalog(); // create only once
        }
        return instance;
    }
    // Information Expert: catalog has all ProductDescriptions
    public ProductDescription getSpecification(ItemID itemID) {
        if (!specifications.containsKey(itemID)) {
            throw new InvalidItemException('Item not found: ' + itemID);
        }
        return specifications.get(itemID);
    }
    public void addSpecification(ProductDescription desc) {
        specifications.put(desc.getItemID(), desc);
    }
    private void loadFromDatabase() {
        // Load product data from persistent storage
    }
}
```

Usage — Accessing Singleton in Register:

```
public class Register {
    private ProductCatalog catalog;

    public Register() {
        // Global visibility — access singleton instance
        this.catalog = ProductCatalog.getInstance();
    }
}
```

SECTION 5: Classes in Code — Inheritance, Interfaces, Abstract Classes

Exam Ref: 2082 Q13b [5], 2081 Ashwin Q13 [5+5], 2080 Chaitra Q13, 2079 Chaitra Q13, 2078 Chaitra Q13, 2077 Chaitra Q13 [5+5]

Q10. Explain the implementation of Inheritance, Abstract Classes, and Interfaces in Java with examples from the POS/OOAD domain. [10] (2081 Ashwin, 2077 Chaitra)

Answer:

1. Inheritance (Generalization) — Java: extends

Inheritance allows a subclass to inherit all attributes and methods of a superclass. In UML, shown as a solid line with a hollow triangle pointing to the parent. In Java, implemented with the 'extends' keyword.

Example — Payment Hierarchy:

```
// Parent (Superclass) — Payment
public class Payment {
    protected Money amount;  // protected so subclasses can access

    public Payment(Money amount) {
        this.amount = amount;
    }

    public Money getAmount() { return amount; }
}
```

```
// Child (Subclass) — CashPayment inherits from Payment
public class CashPayment extends Payment {
    private Money cashTendered;

    public CashPayment(Money cashTendered) {
        super(cashTendered);  // MUST call parent constructor first
        this.cashTendered = cashTendered;
    }

    public Money calculateChange(Money saleTotal) {
        return cashTendered.minus(saleTotal);
    }
}
```

```
// Another Child — CardPayment inherits from Payment
public class CardPayment extends Payment {
    private String cardNumber;
    private String authorizationCode;

    public CardPayment(Money amount, String cardNumber) {
        super(amount);
        this.cardNumber = cardNumber;
    }

    public boolean authorize() {
        // Call payment gateway for authorization
        return true; // simplified
    }
}
```

2. Abstract Classes — Java: abstract keyword

An abstract class cannot be instantiated directly. It may contain both abstract methods (no body, must be overridden) and concrete methods (with body, inherited). Used when all subclasses share some common behavior but vary in other behavior.

```
public abstract class Payment {
    protected Money amount;

    public Payment(Money amount) { this.amount = amount; }

    public Money getAmount() { return amount; } // CONCRETE method — inherited

    // ABSTRACT method — subclasses MUST implement this
    public abstract boolean pay();

    // ABSTRACT method — subclasses MUST implement this
    public abstract String getPaymentType();
}
```

```
public class CashPayment extends Payment {
    public CashPayment(Money amount) { super(amount); }

    @Override
    public boolean pay() {
        return true; // Cash always succeeds
    }

    @Override
    public String getPaymentType() { return 'CASH'; }
}
```

3. Interfaces — Java: interface and implements

An interface defines a contract — a set of method signatures that implementing classes must provide. A class can implement multiple interfaces (Java doesn't allow multiple inheritance of classes). In UML: dashed line + hollow triangle (realization).

```
// Define an interface
public interface IRefundable {
    Money refund(Money refundAmount); // must be implemented by any class
    boolean canRefund();
}

public interface IPrintable {
    void print(Printer printer);
}

// Class implements multiple interfaces
public class CardPayment extends Payment implements IRefundable, IPrintable {
    public CardPayment(Money amount) { super(amount); }

    @Override public boolean pay() { return authorize(); }
    @Override public String getPaymentType() { return 'CARD'; }

    // Implementing IRefundable
    @Override
    public Money refund(Money refundAmount) {
        // Process card refund via payment gateway
        return refundAmount;
    }

    @Override
    public boolean canRefund() { return true; }

    // Implementing IPrintable
    @Override
    public void print(Printer printer) {
        printer.printLine('Card Payment: ' + amount);
    }
```

```
}
```

Abstract Class vs. Interface — Summary:

Aspect	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Method bodies	Can have both abstract and concrete methods	All methods abstract (Java 7); default methods allowed (Java 8+)
Attributes	Can have any attributes (private, protected, etc.)	Only public static final constants
Constructors	Can have constructors	Cannot have constructors
Multiple inheritance	A class can extend only ONE abstract class	A class can implement MULTIPLE interfaces
Use when	Shared implementation among related classes	Unrelated classes need same behavior contract
POS Example	Payment (abstract) — CashPayment, CardPayment share getAmount()	IRefundable — CardPayment, EWalletPayment both implement refund()

SECTION 6: Exception and Error Handling

Exam Ref: 2082 Q14 [10], 2081 Chaitra Q15 [10], 2081 Ashwin Q14 [10], 2080 Chaitra Q14 [10], 2080 Ashwin Q14 [10], 2079 Ashwin Q14 [10], 2079 Chaitra Q14 [10], 2078 Chaitra Q14 [10], 2078 Baishakh Q14 [10], 2077 Chaitra Q14 [10]

Q11. What is exception handling? Why is it important in OO programming? Explain the exception handling mechanism in Java with syntax and examples. [10] (2082 Kartik, 2081 Chaitra, 2080 Chaitra — MOST REPEATED)

Answer:

Exception — Definition

An exception is an abnormal condition or error that disrupts the normal flow of a program's execution. Exception handling is the mechanism by which a program detects, reports, and handles such abnormal conditions gracefully — without crashing.

Why Exception Handling is Important:

- Prevents program crashes — handles errors gracefully rather than abrupt termination.
- Separates error-handling code from normal logic — keeps code clean and readable.
- Provides meaningful error messages to users rather than cryptic system errors.
- Allows recovery — program can attempt alternative actions after an error.
- Enforces robust programming — Java forces handling of checked exceptions.
- Logging and auditing — exceptions can be logged for debugging and analysis.

Java Exception Handling — Key Keywords:

Keyword	Purpose	Usage
try	Encloses code that might throw an exception.	try { risky code here; }
catch	Handles a specific type of exception.	catch (ExceptionType e) { handle it; }
finally	Always executes — whether exception occurs or not. Used for cleanup.	finally { close resources; }
throw	Explicitly throws an exception object.	throw new InvalidItemException('Item not found');
throws	Declares that a method might throw an exception (in method signature).	public void save() throws IOException { ... }

Exception Hierarchy in Java:

Category	Superclass	Examples	Must Catch?
Checked Exceptions	Exception	IOException, SQLException, FileNotFoundException, ClassNotFoundException	YES — must handle or declare with 'throws'
Unchecked Exceptions (Runtime)	RuntimeException	NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, ClassCastException	NO — optional, but good practice
Errors	Error	OutOfMemoryError, StackOverflowError, AssertionError	NO — usually unrecoverable system errors

Basic Exception Handling — Syntax:

```
try {
    // Code that might throw an exception
    ProductDescription spec = catalog.getSpecification(itemID);
    currentSale.makeLineItem(spec, qty);
```

```
} catch (InvalidItemException e) {
    // Handle the specific exception
    System.out.println('Error: Invalid item ID - ' + e.getMessage());
    // Log the error, show error to user, etc.

} catch (NullPointerException e) {
    // Can have multiple catch blocks for different exceptions
    System.out.println('Error: No active sale. Call makeNewSale() first.');

} finally {
    // Always executes — cleanup code
    System.out.println('enterItem attempt completed.');
}
```

Throwing Exceptions — throw and throws:

```
// Method that throws a custom exception
public ProductDescription getSpecification(ItemID itemID) throws InvalidItemException
{
    if (itemID == null) {
        throw new IllegalArgumentException('ItemID cannot be null');
    }
    ProductDescription spec = specifications.get(itemID);
    if (spec == null) {
        // throw statement — explicitly throws an exception
        throw new InvalidItemException('No product found for ID: ' + itemID);
    }
    return spec;
}
```

Q12. What are custom exception classes? Why do we create them? Write a custom exception class for the POS system. Also explain the try-with-resources statement. [10] (2081 Ashwin, 2080 Ashwin, 2079 Chaitra)

Answer:

Custom Exception Classes — Motivation

Custom exception classes are user-defined exception classes that extend Exception (checked) or RuntimeException (unchecked). They are created to represent application-specific error conditions that built-in Java exceptions cannot adequately describe.

Why Create Custom Exceptions:

- Descriptive error names: InvalidItemException is more meaningful than Exception.
- Carry additional error context: Can include itemID, error code, timestamp etc.
- Enable specific catch blocks: Callers can catch exact exception type.
- Enforce domain constraints: e.g., InsufficientStockException, PaymentDeclinedException.
- Better API design: Method signature with 'throws InvalidItemException' is self-documenting.

Custom Checked Exception — InvalidItemException:

```
// Checked exception — callers must handle or declare
public class InvalidItemException extends Exception {
    private ItemID itemID;           // carry domain-specific context
    private int errorCode;

    // Constructor 1: just message
    public InvalidItemException(String message) {
        super(message);
    }

    // Constructor 2: message + cause (chained exceptions)
```

```
    public InvalidItemException(String message, Throwable cause) {
        super(message, cause);
    }

    // Constructor 3: with domain context
    public InvalidItemException(String message, ItemID itemID, int errorCode) {
        super(message);
        this.itemID = itemID;
        this.errorCode = errorCode;
    }

    public ItemID getItemID() { return itemID; }
    public int getErrorCode() { return errorCode; }
}
```

Custom Unchecked Exception — PaymentDeclinedException:

```
// Unchecked exception — extends RuntimeException
public class PaymentDeclinedException extends RuntimeException {
    private String declineReason;

    public PaymentDeclinedException(String message, String reason) {
        super(message);
        this.declineReason = reason;
    }

    public String getDeclineReason() { return declineReason; }
}
```

Custom Exceptions Used in POS System:

```
public void enterItem(ItemID itemID, int qty) {
    try {
        // throws InvalidItemException if item not found
        ProductDescription spec = catalog.getSpecification(itemID);
        if (qty <= 0) {
            throw new IllegalArgumentException('Quantity must be > 0');
        }
        currentSale.makeLineItem(spec, qty);

    } catch (InvalidItemException e) {
        System.err.println('Invalid item [' + e.getItemID() + ']: ' + e.getMessage());
        // Signal UI to show error — don't add item to sale
    }
}
```

try-with-resources Statement (Java 7+):

The try-with-resources statement automatically closes any resource (that implements AutoCloseable) when the try block exits, whether normally or via exception. This eliminates the need for finally blocks for resource cleanup.

```
// WITHOUT try-with-resources (old style)
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader('products.txt'));
    String line = reader.readLine();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (reader != null) {
        try { reader.close(); } catch (IOException e) { e.printStackTrace(); }
    }
}
```

```
// WITH try-with-resources (modern style — auto closes reader)
```

```
try (BufferedReader reader = new BufferedReader(new FileReader('products.txt'))) {
    String line = reader.readLine();
    // process line...
} catch (IOException e) {
    System.err.println('Error reading product file: ' + e.getMessage());
}
// reader.close() automatically called — even if exception occurs
```

Q13. Explain exception propagation and exception chaining in Java. What are checked vs. unchecked exceptions? Explain with examples. [10] (2079 Ashwin, 2078 Chaitra, 2078 Baishakh)

Answer:

Checked vs. Unchecked Exceptions — Detailed Comparison:

Aspect	Checked Exceptions	Unchecked Exceptions (Runtime)
Superclass	extends Exception	extends RuntimeException
Compiler enforcement	Compiler FORCES you to handle or declare	Compiler does NOT require handling
When to use	Recoverable conditions — caller can do something about it	Programming errors, precondition violations — usually bugs
Examples	IOException, SQLException, FileNotFoundException, InvalidItemException (custom)	NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException
Method signature	Must declare: throws IOException	Do not need to declare in signature
POS Example	InvalidItemException — item may not exist in catalog (recoverable)	NullPointerException — currentSale is null (programming bug)

Exception Propagation:

When an exception is thrown and not caught in the current method, it propagates (bubbles up) to the calling method. This continues up the call stack until it is caught or reaches main(), where the JVM terminates the program.

```
// Method C throws exception
public Money getPrice() {
    if (price == null) throw new NullPointerException('Price not set');
    return price;
}

// Method B calls C — doesn't catch — exception propagates UP
public Money getSubtotal() {
    return description.getPrice().times(quantity); // if getPrice() throws, it
propagates
}

// Method A calls B — catches the exception HERE
public Money getTotal() {
    Money total = new Money(0);
    for (SaleLineItem sli : lineItems) {
        try {
            total = total.add(sli.getSubtotal()); // exception may propagate from
getPrice()
        } catch (NullPointerException e) {
            System.err.println('Warning: SaleLineItem has null price — skipping');
        }
    }
    return total;
}
```

Exception Chaining (Wrapping):

Exception chaining (also called exception wrapping) is the practice of catching a low-level exception and re-throwing it as a higher-level, more meaningful exception. The original exception is preserved as the 'cause'. This allows the API to present meaningful domain exceptions while preserving debugging information.

```
public ProductDescription loadFromDB(ItemID itemID) throws InvalidItemException {
    try {
        // Low-level database call
        return database.query('SELECT * FROM products WHERE id = ?', itemID);

    } catch (SQLException e) {
        // Chain: wrap SQLException in domain-level InvalidItemException
        // The original SQLException is preserved as the 'cause'
        throw new InvalidItemException(
            'Failed to load product ' + itemID + ' from database',
            e  // <-- this is the 'cause' — the original exception
        );
    }
}

// The caller sees InvalidItemException (domain level)
// But can still access original SQLException via e.getCause()
```

SECTION 7: Complete Implementation Case Studies

Exam Ref: 2082 Q15 [10], 2081 Ashwin Q15, 2080 Chaitra Q15, 2079 Chaitra Q15, 2078 Baishakh Q15, 2077 Chaitra Q15

Q14. Write complete Java class implementations for the Library Management System covering: Book, Member, Loan, Fine, LibraryController. Include constructors, methods, and exception handling. [10] (2082 Kartik, 2081 Chaitra Q15)

Answer:

Custom Exceptions — Library System:

```
public class BookNotFoundException extends Exception {
    public BookNotFoundException(String isbn) {
        super('Book not found: ' + isbn);
    }
}

public class BookNotAvailableException extends Exception {
    public BookNotAvailableException(String isbn) {
        super('No copies available for: ' + isbn);
    }
}

public class MemberNotFoundException extends Exception {
    public MemberNotFoundException(String memberID) {
        super('Member not found: ' + memberID);
    }
}
```

Java Class — Book:

```
import java.util.ArrayList;
import java.util.List;

public class Book {
    private String isbn;
    private String title;
    private String author;
    private int totalCopies;
    private int availableCopies;

    public Book(String isbn, String title, String author, int copies) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
        this.totalCopies = copies;
        this.availableCopies = copies;
    }

    public boolean isAvailable() {
        return availableCopies > 0;
    }

    public void issueOneCopy() throws BookNotAvailableException {
        if (!isAvailable()) {
            throw new BookNotAvailableException(isbn);
        }
        availableCopies--;
    }

    public void returnOneCopy() {
        if (availableCopies < totalCopies) {
            availableCopies++;
        }
```

```
    }

    // Getters
    public String getIsbn() { return isbn; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public int getAvailableCopies() { return availableCopies; }
}
```

Java Class — Member:

```
public class Member {
    private String memberID;
    private String name;
    private String email;
    private int maxBooksAllowed;
    private List<Loan> activeLoans = new ArrayList<>();

    public Member(String memberID, String name, String email) {
        this.memberID = memberID;
        this.name = name;
        this.email = email;
        this.maxBooksAllowed = 5; // default
    }

    public boolean canBorrow() {
        return activeLoans.size() < maxBooksAllowed;
    }

    public void addLoan(Loan loan) { activeLoans.add(loan); }
    public void removeLoan(Loan loan) { activeLoans.remove(loan); }
    public List<Loan> getActiveLoans() { return activeLoans; }
    public String getMemberID() { return memberID; }
    public String getName() { return name; }
}
```

Java Class — Loan:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class Loan {
    private String loanID;
    private Book book;
    private Member member;
    private LocalDate issueDate;
    private LocalDate dueDate;
    private LocalDate returnDate;
    private boolean returned;
    private static int loanCounter = 0;

    public Loan(Book book, Member member) {
        this.loanID = 'LN' + (++loanCounter);
        this.book = book;
        this.member = member;
        this.issueDate = LocalDate.now();
        this.dueDate = issueDate.plusDays(14); // 2 weeks loan period
        this.returned = false;
    }

    public boolean isOverdue() {
        return !returned && LocalDate.now().isAfter(dueDate);
    }

    public long getDaysOverdue() {
```

```
        if (!isOverdue()) return 0;
        return ChronoUnit.DAYS.between(dueDate, LocalDate.now());
    }

    public void markReturned() {
        this.returned = true;
        this.returnDate = LocalDate.now();
        book.returnOneCopy(); // update book availability
    }

    public String getLoanID() { return loanID; }
    public Book getBook() { return book; }
    public Member getMember() { return member; }
    public LocalDate getDueDate() { return dueDate; }
}
```

Java Class — Fine:

```
public class Fine {
    private static final double FINE_PER_DAY = 5.0; // Rs. 5 per day
    private String fineID;
    private Loan loan;
    private double amount;
    private boolean paid;
    private static int fineCounter = 0;

    public Fine(Loan loan) {
        this.fineID = 'FN' + (++fineCounter);
        this.loan = loan;
        // Information Expert: Fine calculates its own amount
        this.amount = loan.getDaysOverdue() * FINE_PER_DAY;
        this.paid = false;
    }

    public void pay() {
        if (paid) {
            throw new IllegalStateException('Fine ' + fineID + ' already paid');
        }
        this.paid = true;
    }

    public double getAmount() { return amount; }
    public boolean isPaid() { return paid; }
    public String getFineID() { return fineID; }
}
```

Java Class — LibraryController (Facade Controller):

```
import java.util.HashMap;
import java.util.Map;

public class LibraryController {
    private Map<String, Book> books = new HashMap<>();
    private Map<String, Member> members = new HashMap<>();
    private Map<String, Loan> activeLoans = new HashMap<>();

    // Add a book to the library catalog
    public void addBook(Book book) {
        books.put(book.getIsbn(), book);
    }

    // Register a new member
    public void registerMember(Member member) {
        members.put(member.getMemberID(), member);
    }
```

```
    // Issue a book — Controller handles system operation
    public Loan issueBook(String isbn, String memberID)
            throws BookNotFoundException, BookNotAvailableException,
MemberNotFoundException {

        // Validate inputs — throw if not found
        Book book = books.get(isbn);
        if (book == null) throw new BookNotFoundException(isbn);

        Member member = members.get(memberID);
        if (member == null) throw new MemberNotFoundException(memberID);

        if (!member.canBorrow()) {
            throw new IllegalStateException('Member has reached borrow limit');
        }

        book.issueOneCopy(); // throws BookNotAvailableException if unavailable

        // Creator: Controller creates Loan (records loans)
        Loan loan = new Loan(book, member);
        activeLoans.put(loan.getLoanID(), loan);
        member.addLoan(loan);
        return loan;
    }

    // Return a book — may generate Fine
    public Fine returnBook(String loanID) {
        Loan loan = activeLoans.get(loanID);
        if (loan == null) {
            throw new IllegalArgumentException('Loan not found: ' + loanID);
        }

        loan.markReturned();
        activeLoans.remove(loanID);
        loan.getMember().removeLoan(loan);

        // Generate fine if overdue
        if (loan.isOverdue()) {
            return new Fine(loan); // Creator: Controller creates Fine
        }
        return null; // no fine if returned on time
    }
}
```

SECTION 8: Additional Implementation Topics

Exam Ref: 2082 Q10b [3], 2081 Ashwin Q10 [5+5], 2080 Ashwin Q10, 2079 Ashwin Q10, 2078 Chaitra Q10, 2077 Chaitra Q10 [5+5]

Q15. What is a Design Pattern? Explain the Singleton and Factory Method patterns with Java code. [5+5] (2081 Ashwin, 2077 Chaitra)

Answer:

Design Patterns — Overview

A Design Pattern is a reusable solution to a commonly occurring design problem. Patterns are not complete code — they are templates/blueprints that describe how to solve a problem in a general way. GoF (Gang of Four) patterns are the most well-known, categorized into Creational, Structural, and Behavioral.

Category	Description	Examples
Creational Patterns	Concerned with object creation mechanisms.	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural Patterns	Deal with object composition and structure.	Adapter, Bridge, Composite, Decorator, Facade, Proxy
Behavioral Patterns	Deal with communication between objects.	Observer, Strategy, Command, Iterator, Template Method

Singleton Pattern — (See Section 4 for full code)

Ensures a class has only ONE instance. Key elements: private static instance, private constructor, public static getInstance(). Used for: ProductCatalog, Logger, Configuration.

Factory Method Pattern:

The Factory Method pattern defines an interface for creating an object, but lets subclasses decide which class to instantiate. Instead of calling 'new' directly, a factory method is called.

```
// Product interface
public interface Payment {
    boolean pay(double amount);
}

// Concrete products
public class CashPayment implements Payment {
    @Override public boolean pay(double amount) {
        System.out.println('Paid Rs.' + amount + ' in cash');
        return true;
    }
}

public class CardPayment implements Payment {
    @Override public boolean pay(double amount) {
        System.out.println('Charging Rs.' + amount + ' to card');
        return authorizeWithBank(amount);
    }
    private boolean authorizeWithBank(double amount) { return true; }
}

// Factory class — creates Payment objects without exposing concrete class
public class PaymentFactory {
    public static Payment createPayment(String type) {
        switch (type.toUpperCase()) {
            case 'CASH': return new CashPayment();
            case 'CARD': return new CardPayment();
            case 'WALLET': return new WalletPayment();
```

```
            default: throw new IllegalArgumentException('Unknown payment type: ' +
type);
        }
    }
}

// Client code — uses factory instead of 'new'
Payment payment = PaymentFactory.createPayment('CARD');
payment.pay(1500.0);
```

Factory Method vs. Singleton — Key Differences:

Aspect	Singleton	Factory Method
Category	Creational	Creational
Purpose	Restrict class to one instance	Delegate object creation to subclasses/factory
Instance count	Exactly ONE	Creates NEW instance each call (or managed)
Key method	getInstance()	createObject() / createPayment()
Benefit	Shared global state, saves memory	Decouples creation from usage; easy to extend
POS Example	ProductCatalog.getInstance()	PaymentFactory.createPayment('CASH')

Q16. Explain the Observer Pattern. Give an example in the context of an OO system. How does it support Low Coupling? [5] (2082 Kartik Q10b, 2080 Chaitra)

Answer:

Observer Pattern — Definition

The Observer Pattern defines a one-to-many dependency between objects. When the 'Subject' (Observable) changes state, all its registered 'Observers' are automatically notified. Also called the Publish-Subscribe pattern.

Components:

Component	Role	Example (POS)
Subject (Observable)	Maintains list of observers. Notifies them on state change.	Sale — notifies observers when sale is complete.
Observer (Interface)	Defines the update() method all observers must implement.	ISaleObserver with onSaleComplete(Sale sale) method.
Concrete Observer	Implements the update() method to react to change.	InventorySystem, ReceiptPrinter, SalesLogger

```
// Observer interface
public interface ISaleObserver {
    void onSaleComplete(Sale sale);
}

// Concrete observers
public class InventorySystem implements ISaleObserver {
    @Override
    public void onSaleComplete(Sale sale) {
        // Reduce stock for each sold item
        for (SaleLineItem sli : sale.getLineItems()) {
            inventory.reduceStock(sli.getDescription().getItemID(),
sli.getQuantity());
        }
    }
}
```

```
// Subject — Sale notifies observers when complete
public class Sale {
    private List<ISaleObserver> observers = new ArrayList<>();

    public void addObserver(ISaleObserver obs) { observers.add(obs); }

    public void becomeComplete() {
        this.isComplete = true;
        // Notify all registered observers
        for (ISaleObserver obs : observers) {
            obs.onSaleComplete(this);
        }
    }
}
```

Observer supports LOW COUPLING because Sale doesn't directly know about InventorySystem or ReceiptPrinter. It only knows about the ISaleObserver interface. New observers can be added without modifying Sale — Open/Closed Principle.

QUICK REVISION: Chapter 4 — Key Points Summary

Topic	Key Points to Remember	Expected Marks
Mapping Design to Code	DCD → class skeletons. Collab Diagram → method bodies. Each message = one statement. Sequence numbers = code order.	4–5
Mapping Rules (DCD → Java)	- attr = private field. + attr = public field. # = protected. extends = generalization. implements = realization. 1..* = List<T>.	5–8
Register class	Facade Controller. Attributes: currentSale, catalog. Methods: makeNewSale(), enterItem(), endSale(), makePayment().	8–10
Sale class	Key domain class. makeLineItem() = Creator. getTotal() = Information Expert. makePayment() = Creator of Payment.	8–10
SaleLineItem class	getSubtotal() = qty * price. Uses ProductDescription (parameter/attribute visibility).	4–6
Payment hierarchy	Abstract Payment class. CashPayment extends Payment. CardPayment extends Payment. Polymorphism applied.	5–8
ProductCatalog Singleton	private static instance. private constructor. public static getInstance(). Contains specifications Map.	4–6
Collab Diagram → Methods	Message 1 = method being written. 1.1 = first line inside. 1.2 = second line. <<create>> = new ClassName(). * = for loop.	8–10
Updating Class Definitions	Add missing attributes (taxRate, store). Refine types. Add private helpers. Add collection types (ArrayList). Singleton implementation.	3–5
Inheritance (extends)	Child calls super() in constructor. @Override for overridden methods. All parent methods/fields inherited.	5–8
Abstract class	Cannot instantiate. Has abstract methods (no body). Has concrete methods (with body). Used when subclasses share behavior.	5–8
Interface (implements)	All methods abstract (default Java 7). Class can implement multiple interfaces. Use for unrelated classes needing same contract.	5–8
Exception Keywords	try: risky code. catch: handle specific type. finally: always runs (cleanup). throw: create & throw exception. throws: declare in signature.	5–8
Checked vs Unchecked	Checked (extends Exception): must handle, recoverable (IOException, InvalidItemException). Unchecked (extends RuntimeException): optional, bugs (NPE).	5–8
Custom Exceptions	extends Exception or RuntimeException. Add domain context (itemID, errorCode). Constructor with (message) and (message, cause).	5–8
Exception Propagation	Unhandled exception bubbles up call stack until caught or program ends.	3–4
Exception Chaining	catch (SQLException e) { throw new InvalidItemException(msg, e); } — wraps low-level in domain exception, preserves cause.	4–5
try-with-resources	try (Resource r = new Resource()) { ... } — auto-closes any AutoCloseable. No need for finally to close.	3–4
Factory Method Pattern	PaymentFactory.createPayment('CASH') — decouples creation from usage. switch on type. Returns interface type.	4–6
Observer Pattern	Subject (Sale) notifies Observers (InventorySystem, Printer) via ISaleObserver interface. Low coupling — Sale doesn't know observer types.	4–6